

ManicBot

Полная архитектура проекта

Telegram-бот для записи в салон красоты

Cloudflare Workers • D1 • KV • Stripe • Google Calendar • Workers AI

22 марта 2026

Обзор стека технологий

Компонент	Технология	Назначение
Runtime	Cloudflare Workers	Serverless-функции, обработка webhook
Хранилище KV	Cloudflare KV	Быстрый key-value: состояние, кеш, сессии
База данных	Cloudflare D1 (SQLite)	Реляционные данные: записи, пользователи, тенанты
AI	Workers AI (LLM)	Интеллектуальный ассистент, распознавание намерений
Оплата	Stripe API	Подписки, биллинг, checkout
Календарь	Google Calendar API	OAuth2 + Service Account, двусторонняя синхронизация
Мессенджер	Telegram Bot API	Интерфейс клиентов, мастеров, админов
Тесты	Vitest 4.x + Miniflare	40+ тестовых файлов
Деплой	GitHub Actions → Wrangler	CI/CD pipeline
Админка	Next.js на Vercel	Веб-дашборд для платформы

Зависимости (package.json)

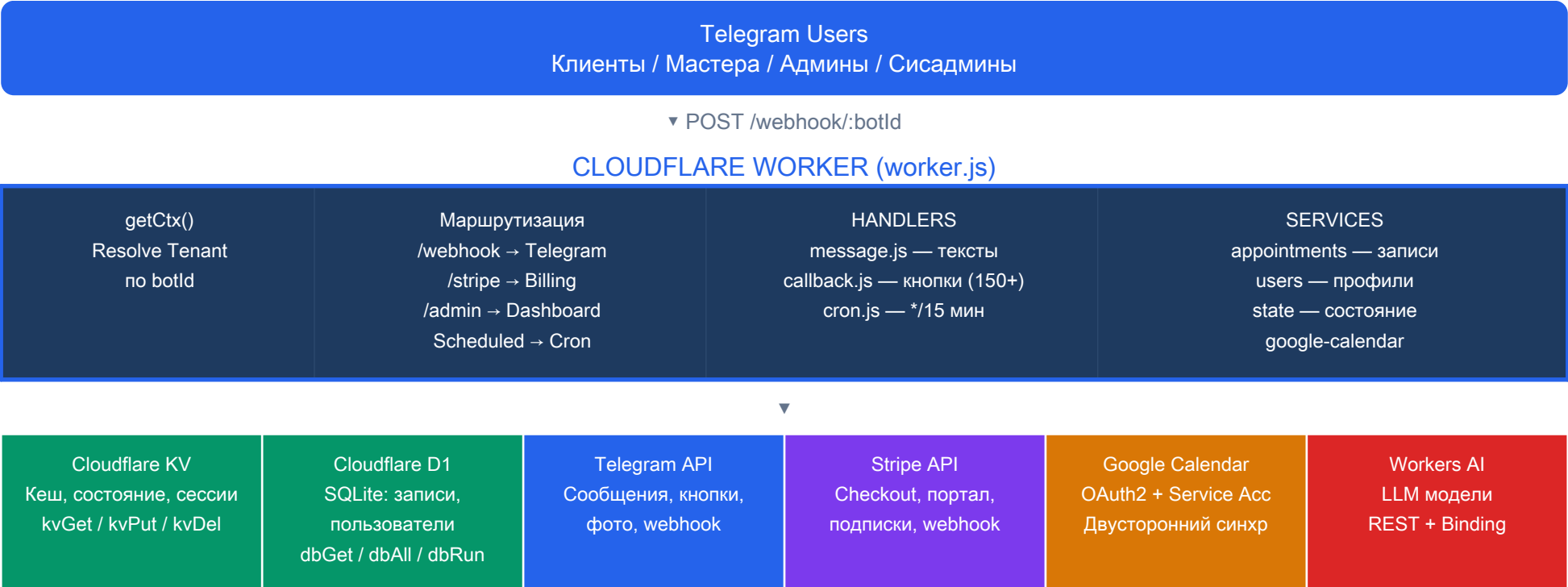
Пакет	Назначение
vitest ^4.0.18	Тестовый фреймворк
wrangler ^4.71.0	Cloudflare CLI для деплоя и разработки
@cloudflare/vitest-pool-workers ^0.12.20	Vitest адаптер для Workers

Runtime-биндинги Cloudflare: MANICBOT (KV) • DB (D1 SQLite) • AI (Workers AI)

Структура файлов проекта

```
manicbot/
├── src/
│   ├── worker.js — Точка входа, маршрутизация
│   ├── config.js — Константы: CB, STEP
│   ├── ai.js — AI промпты, парсинг
│   ├── telegram.js — Telegram API
│   ├── patterns.js — Regex намерения
│   ├── notifications.js — Уведомления
│
│   ├── handlers/
│   │   ├── message.js — Текстовые сообщения
│   │   ├── callback.js — Кнопки (150+)
│   │   └── cron.js — Каждые 15 мин
│
│   └── services/
│       ├── users.js — Пользователи, роли
│       ├── appointments.js — Записи, слоты
│       ├── chat.js — История чата
│       ├── state.js — Состояние (KV)
│       ├── services.js — Услуги салона
│       ├── tickets.js — Тикеты
│       ├── google-cal*.js — OAuth2, синх
│       └── calendar.js — Google Cal API
│
│   ├── roles/roles.js — Роли
│
│   ├── tenant/
│   │   ├── resolver.js — Определение tenant
│   │   ├── storage.js — Реестр (D1+KV)
│   │   └── migration.js — Legacy → multi
│
│   ├── billing/
│   │   ├── config.js — START/PRO/STUDIO
│   │   ├── features.js — Рейтинг фич
│   │   ├── lifecycle.js — Trial, grace
│   │   ├── stripe.js — Checkout, портал
│   │   └── webhooks.js — Stripe webhook
│
│   ├── ui/
│   │   ├── screens.js — Экраны
│   │   ├── booking.js — Процесс записи
│   │   ├── keyboards.js — Inline-клавиатуры
│   │   ├── admin.js — Админ-панель
│   │   └── sysadmin.js — Платформа
│
│   ├── i18n/ (RU, UA, EN, PL × 12 модулей)
│   └── utils/ (kv, db, date, security, helpers, ics)
│
├── test/ 40+ тестов (Vitest)
├── admin-app/ Next.js дашборд
├── scripts/ Миграция, провизининг
└── wrangler.toml CF конфиг
```

Архитектура системы



Мультитенантность

1 Salon = 1 Tenant = 1+ Bots

Tenant ID t_<random> KV ключи: t:{tenantId}... D1 таблицы: tenants, bots, tenant_roles	Резолюция ролей 1. Platform roles (D1: platform_roles) ↓ 2. Tenant roles (D1: tenant_roles) ↓ 3. Client (по умолчанию) Функция: resolveRole(ctx, chatId)	Контекст (ctx) buildTenantCtx(env, resolved): • ctx.kv = KV namespace • ctx.db = D1 database • ctx.prefix = "t:{id}:" • ctx.tenantId • ctx.botToken • ...env vars
--	---	---

Таблица ролей

Уровень	Роль	Доступ
Платформа	system_admin	Полный доступ ко всей платформе: тенанты, боты, агенты
Платформа	support	Тикеты поддержки со всех салонов
Платформа	technical_support	Технические тикеты
Тенант	tenant_owner	Владелец салона: мастера, услуги, настройки, биллинг
Тенант	master	Мастер: подтверждение записей, календарь, отпуск
По умолч.	client	Клиент: запись, просмотр, поддержка

Биллинг и подписки (Stripe)

План	Мастеров	AI	Поддержка	Календарь	White-label
START	1	—	—	—	—
PRO	5	✓	✓	✓	—
STUDIO	∞	✓	✓	✓	✓

Жизненный цикл подписки



Stripe интеграция

Компонент	Описание
Webhook	/stripe/webhook — подпись верифицируется через HMAC-SHA256
События	subscription.updated, subscription.deleted, invoice.payment_failed
Checkout	Stripe Checkout Session → Подписка → D1 обновление
Портал	Customer Portal для управления подпиской
Feature gating	canUse(ctx, feature) — проверка доступа по плану

Процесс записи (Booking Flow)

№	Шаг	Описание	Файл
1	Выбор услуги	Клиент нажимает "Записаться" → список услуг (classic, gel, pedi, ext, design, combo)	ui/booking.js
2	Выбор даты	Календарь с доступными днями (30 дней вперёд)	ui/keyboards.js
3	Выбор времени	Слоты по 30 мин (9:00–19:00). Учёт: записи, отпуск, Google Calendar	services/appointments.js
4	Подтверждение	Статус: pending. Уведомление мастерам	notifications.js
5	Реакция мастера	Кнопки: Подтвердить / Отклонить / Предложить другое время	handlers/callback.js
6	Подтверждено	Статус: confirmed. Google Calendar event. Клиенту сообщение	services/calendar.js
7	Напоминания	Cron: за 24ч и за 2ч до записи	handlers/cron.js

Статусы записи

Статус	Значение	Переходы
pending	Новая, ждёт подтверждения мастера	→ confirmed / rejected / counter / cancelled
confirmed	Подтверждена мастером	→ cancelled
counter	Мастер предложил другое время	→ confirmed / cancelled
rejected	Отклонена мастером	Конечный статус
cancelled	Отменена клиентом/админом	Конечный статус

Услуги по умолчанию

ID	Емоји	Длит. (мин)	Цена
classic	☐f485	60	80
gel	☐f48e	90	140
pedi	☐f9b6	90	120
ext	☐	120	250
design	☐f3a8	30	50
combo	☐f451	150	220

AI-интеграция (Workers AI)

Цепочка моделей (fallback)

Приоритет	Модель	Назначение
1 (основная)	@cf/openai/gpt-oss-120b	Основная модель, наилучшее качество
2 (запасная)	@cf/meta/llama-4-scout-17b	Быстрая модель Meta
3 (резерв)	@cf/meta/llama-3.1-8b-instruct	Лёгкая модель, минимальная латентность

Как работает AI

№	Шаг	Описание
1	Пользователь пишет текст	Текстовое сообщение в Telegram
2	Regex-паттерны	50+ regex проверяют намерение (RU, UA, EN, PL)
3	AI анализ	Если regex не совпал → Workers AI с контекстом салона
4	Теги действий	AI возвращает теги: [BOOK:classic], [MY_APTS]...
5	Исполнение	executeAIAction() превращает теги в действия бота

Основные AI-действия

Тег	Действие
[MY_APTS]	Показать записи клиента
[BOOK:svcId:date:time]	Начать запись на услугу

Ter	Действие
[PRICES]	Показать прайс-лист
[CONTACTS]	Контакты салона (Instagram, телефон)
[CANCEL_ALL]	Отменить все записи клиента
[CONSULT]	Запросить консультанта
[ADM_PANEL]	Открыть админ-панель
[MST_CALENDAR]	Настройки календаря мастера
[SYSADM_PANEL]	Платформенный админ
[LANG:xx]	Сменить язык интерфейса

Google Calendar интеграция

Service Account	OAuth2
Мастер даёт Calendar ID Бот использует сервисный аккаунт	Мастер авторизует через Google Бот хранит зашифрованный refresh token
Простая настройка Нужен доступ к календарю	Безопасный режим AES-GCM шифрование токенов в D1

Двусторонняя синхронизация

Направление	Триггер	Действие
Исходящая →	Подтверждение записи	Создание Google Calendar event
Исходящая →	Изменение записи	Обновление / удаление event
← Входящая	Cron (* / 15 мин)	Проверка busy-блоков → исключение из слотов
← Входящая	Обновление watch	Авто-продление каждые ~2.5 дня

OAuth2 Flow

№	Шаг
1	Мастер нажимает "Подключить Google Calendar"
2	Редирект на Google OAuth consent screen
3	Callback → сохранение refresh token (зашифрованного в D1)

№	Шаг
4	Выбор календаря из списка доступных
5	Двусторонняя синхронизация активна

Хранилище данных

KV Namespace — быстрый кеш и состояние

Ключ	TTL	Данные
t:{id}:state:{chatId}	2ч	Шаг диалога (машина состояний)
t:{id}:chat:{chatId}	1ч	История чата (8 сообщений для AI)
t:{id}:rl:{chatId}	60с	Rate limit счётчик
t:{id}:lang:{chatId}	∞	Язык пользователя (ru/ua/en/pl)
t:{id}:master:{chatId}	∞	Профиль мастера (JSON)
t:{id}:ap:{aptId}	∞	Документ записи (JSON)
t:{id}:all:YYYY-MM	∞	Список aptId за месяц
t:{id}:d:YYYY-MM-DD	∞	Список aptId за день
t:{id}:u:{chatId}	∞	Профиль пользователя (JSON)
t:{id}:blocked:{chatId}	∞	Маркер заблокированного

D1 Database — реляционные данные

Таблица	Назначение	Ключевые поля
tenants	Салоны	id, name, plan, billingStatus, stripeCustomerId
bots	Реестр ботов	bot_id, tenant_id, bot_token (encrypted), active
platform_roles	Глобальные роли	chat_id, role

Таблица	Назначение	Ключевые поля
tenant_roles	Тенантные роли	tenant_id, chat_id, role
appointments	Записи	id, tenant_id, chat_id, svc_id, date, time, status
masters	Мастера	tenant_id, chat_id, name, services, work_hours
services	Услуги	tenant_id, service_id, name, price, duration
google_integrations	Google Calendar	id, tenant_id, calendar_id, refresh_token_enc
google_busy_blocks	Занятые блоки	integration_id, event_id, start_ts, end_ts
local_tickets	Тикеты в салоне	tenant_id, client_cid, master_cid, open

Интернационализация (i18n)

Русский ru	Українська ua	English en	Polski pl
---------------	------------------	---------------	--------------

12 модулей на каждый язык

Модуль	Назначение
admin.js	Строки админ-панели
billing.js	Строки биллинга и оплаты
booking.js	Процесс записи
gcal.js	Google Calendar строки
general.js	Общие сообщения
master.js	Строки для мастеров
menu.js	Метки меню
meta.js	Мета: дни недели, месяцы
screens.js	Заголовки экранов
services.js	Названия услуг
support.js	Тикеты поддержки
sysadmin.js	Платформенный админ

Механизм

`t(lang, key)` → поиск в модуле языка → fallback на русский

`detectLang(text)` → автоопределение по первому сообщению

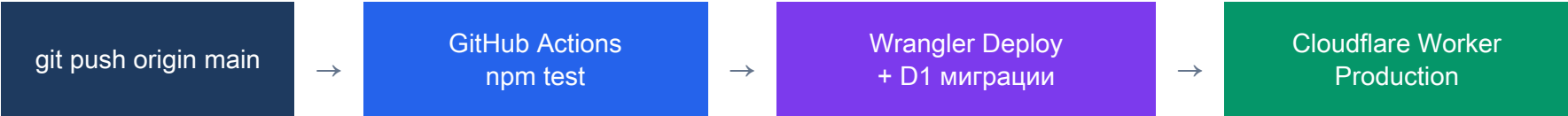
`fill(template, vars)` → подстановка переменных в шаблоны

Тестирование и деплой

Тесты (Vitest 4.x, 40+ файлов)

Категория	Файлы	Что тестирует
Записи	appointments-flow, calendar-sync	Весь жизненный цикл записи
Кнопки	button-logic, all-roles-callbacks	150+ callback обработчиков
AI	ai-callbacks, ai-tenant-context	Парсинг AI действий
Биллинг	billing-features, lifecycle, webhooks	Stripe, планы, фичи
Роли	routing-all-roles, sysadmin-protection	Доступ по ролям
i18n	i18n-integrity	Все языки содержат все ключи
Тенанты	tenant-resolver, tenant	Мультитенантность
Безопасность	security, worker-no-hardcoded-tokens	Шифрование, токены
Паттерны	patterns, patterns-extended	Regex для намерений

CI/CD Pipeline



Cron (каждые 15 минут)

Фаза	Действие
Phase 1	Напоминания о записях (за 24ч и за 2ч)
Phase 2	Retry синхронизации Google Calendar (если нет event_id)
Phase 3	Очистка отменённых записей (старше 48ч)

Маршрутизация запросов (worker.js)

Метод	Путь	Обработчик
POST	/webhook/:botId	Telegram webhook (мультибот)
POST	/stripe/webhook	Stripe подписки, оплаты
GET	/stripe/success	Успешная оплата (redirect)
GET	/admin/migrate	Миграция legacy → multi-tenant
GET	/admin/seed	Демо-данные для тестирования
GET	/setup	Регистрация Telegram webhook
GET	/remove-webhook	Удаление webhook
GET	/admin	HTML админ-панель (Basic Auth)
GET	/admin/billing	Страница биллинга
GET	/calendar/:aptId.ics	Скачать ICS файл
GET	/google/callback	Google OAuth2 callback
GET	/google/select	Выбор календаря Google
Scheduled	* / 15 * * * *	handleCron(): напоминания, синх, очистка

Обработка сообщения (message.js)

№	Шаг
1	Пользователь отправляет текст в Telegram
2	Telegram webhook → POST /webhook/{botId}

№	Шаг
3	worker.js → getCtx() → resolve tenant
4	onMsg(ctx, message) → обработка
5	Pattern detection: regex (50+) → если нет → AI
6	AI возвращает теги: [MY_APTS], [BOOK:...]
7	executeAIAction() → вызов сервисов
8	Ответ через Telegram API + сохранение истории

Секреты и конфигурация

Secrets (wrangler secret put)

Переменная	Назначение
BOT_TOKEN	Telegram bot token
ADMIN_KEY	HTTP Basic Auth пароль
WEBHOOK_SECRET	Telegram webhook secret
STRIPE_SECRET_KEY	Stripe API ключ
STRIPE_WEBHOOK_SECRET	Stripe webhook подпись (HMAC)
STRIPE_PRICE_*_MONTHLY	Price ID для планов START/PRO/STUDIO
CLOUDFLARE_ACCOUNT_ID	Workers AI REST API
WORKERS_AI_API_TOKEN	Workers AI токен
GOOGLE_SERVICE_ACCOUNT_KEY	Google Calendar (base64 JSON)
GOOGLE_OAUTH_CLIENT_ID	Google OAuth2 Client ID
GOOGLE_OAUTH_CLIENT_SECRET	Google OAuth2 Client Secret
GOOGLE_TOKEN_ENCRYPTION_KEY	Шифрование refresh token (AES-GCM)
BOT_ENCRYPTION_KEY	Шифрование bot token в KV

Environment Variables (wrangler.toml)

Переменная	Назначение
APP_BASE_URL	Базовый URL для callbacks
LANDING_URL	Лендинг (Cloudflare Pages)
ADMIN_APP_URL	Админка (Cloudflare Pages)
ADMIN_CHAT_ID	Telegram ID платформенного админа

Cloudflare Bindings (wrangler.toml)

Тип	Имя	ID
KV Namespace	MANICBOT	62a7d16805e742918a82184e879537cc
D1 Database	DB (manicbot-db)	2c9bfdad-3dc0-4290-b26d-bcada6c02bfb
Workers AI	AI	Built-in binding